

HelixSkills

HelixSkills

Регулисан, уставно заснован систем вештина за агенте CLI AI.

Сажетак

HelixSkills је систем вештина за агенте CLI AI који наслеђује Helix Constitution као подмодул, тако да се свако универзално правило управљања примењује безусловно. Обухвата инсталабилне вештине агената, MCP серверске алате, Claude додатке за Цоде и поново употребљиве моторе иза регистроване, документоване каталога.

Кратак опис

HelixSkills је систем вештина за агенте CLI AI. Уграђује Helix Constitution као подмодул како би сва универзална правила била на снази, а затим испоручује регистроване вештине (акциони префикс, валидатор медија, мултитрак, синхронизацију сесија, животни циклус радних јединица и још много тога), два MCP серверска алата, два Claude додатка за Цоде и поново употребљиве моторе.

Детаљан опис

HelixSkills (репозиторијум skills, Апацхе-2.0) је систем вештина за агенте CLI AI, а почиње од намерног обрнутог приступа уобичајеном редоследу: управљање долази пре могућности. Наслеђује Helix Constitution као свој constitution/ подмодул, тако да се свако универзално правило из constitution/CLAUDE.md и constitution/Constitution.md примењује безусловно — не као конвенција коју агент може поштовати,

већ као скуп правила физички уграђених у структуру пројекта. Агент који усвоји HelixSkills не може се одрећи устава; правила путују заједно са кодом.

Док већина „оквира за вештине” тргује апстракцијама, HelixSkills испоручује конкретан, регистровани инвентар на који можете указати и инсталирати га. Седам уставних вештина инсталира се преко `register.sh`: `ацион-prefix-system`, `медиа-валидатор`, `мултитрацк`, `репортинг-workable-итемс`, `сцхедулед-work-queue`, `сесион-sync` и `workable-итем-lifecycle` — спектар оцењен од средњег до напредног нивоа, који покрива све од дисциплинованог именовања акција до валидације медија и пуног животног циклуса радне јединице. Додатне вештине у развоју (Android преглед, Java/Kotlin језик, Linux OS) већ су индексирани и припремљени за активацију. Два MCP серверска алата (медиа-валидатор, `сцхедулед-work`) излажу те вештине агентима преко Model Context Protocol, док два Claude додатка за Цоде (`helix`, `сцхедулед-work`) убацују исте могућности директно у радно окружење агента — један скуп вештина, доступан агентима без обзира на то којим интерфејсом комуницирају.

Испод каталога налазе се четири поново употребљива мотора првог нивоа — континуум (имплементиран), плус `сесион_орцхестратор`, `токен_оптимизер` и `цлицкуп_sync` (у фази дизајна) — заједничка инфраструктура која спречава да вештине поново измишљају исту основну логику. Само `токен_оптимизер` декларише експлицитан граф зависности који се протеже до пакета екосистема `vasic-digital` (ТООН, Ембеддингс, VectorDB, Нормализе, `цонверсатион`) и HelixDevelopment-овог LLMProvider, тако да је умрежавање између репозиторијума проверљиво, а не имплицитно. Око свега тога функционише дисциплинована документација: каталог вештина, аутоматски генерисан индекс графа вештина, детаљне странице по репозиторијумима и отворен регистар Недостатака и ризика који јасно наводи шта још није урађено. Цео систем је огледало на GitHub, GitLab, GitFlic и GitVerse ради отпорности и регионалног приступа.

Зашто смо га изградили

Агентима CLI и AI потребне су могућности које су доследне, регулисане и поново употребљиве – а не ад хок скрипте које свака изнова измишљају правила. HelixSkills је створен да агентима пружи пакет регистрованих вештина везаних за заједнички устав, како би понашање остало доследно и проверљиво у сваком агенту и пројекту који га усвоје.

Зашто је револуционарно

Омогућава да способности агента буду преносиве и усклађене с правилима *по самој конструкцији*, а не захваљујући дисциплини. Свака вештина је регулисана, верзионирана и инсталабилна јединица подржана уставним подмодулом – тако да у тренутку када агент региструје вештину, истовремено наслеђује и канонски скуп правила, без могућности одступања. То откључава нешто што раније није било практично: преношење способности с једног агента или пројекта на други уз уверење да она стиже већ везана за исту управу, изложена преко стандардних интерфејса (МСР сервери и Claude Цоде додаци) уместо хрпе прилагођених скрипти које свака изнова дефинишу правила.

Шта је иновативно

- **Constitution као подмодул:** универзална управљачка правила се наслеђују, а не копирају – монтирају се у стабло тако да сваки агент који их користи буде везан за исти канонски скуп правила, с ажурирањима која долазе из једног извора истине уместо из десетак застарелих копија.
- Вештине испоручене као саморегиструјуће јединице (`register.sh`) и увезане у аутоматски генерисан индекс графова вештина, тако да каталог остаје откљив и никада не излази из синхронизације с оним што је заправо инсталирано.
- Излагање на више интерфејса: исти скуп вештина долази до агената преко МСР алата-сервера и Claude Цоде додатака – пише се једном, а користи се у било ком окружењу које агент користи.
- Поново употребљиви мотори првог нивоа (цонтинуум, `token_оптимизер`, `сесион_оркестратор`, `цлицкуп_sync`) дељени широм екосистема, од којих сваки носи експлицитне, проверљиве декларације међузависности уместо скривеног спрежања.

Највећи технички изазови и како смо их решили

- **Одржавање доследног и усклађеног понашања агената кроз многе вештине и агенте** – поновна имплементација управљања по вештини гарантује одступање током времена. Решено монтирањем Helix Constitution као подмодула, тако да правила у `constitution/CLAUDE.md` и `constitution/Constitution.md` важе безусловно и ажурирају се из једног извора, уместо да се копирају и оставе да застаре.

- **Омогућавање инсталације и откривања растућег скупа вештина** – каталог је бескористан ако нико не може да пронађе или инсталира оно што се у њему налази. Решено регистрацијом по вештини помоћу `register.sh`, која повезује сваку вештину при инсталацији, уз аутоматски генерисан INDEX граф вештина и детаљну документацију по репозиторијуму, тако да откривање прати стварност у реалном времену.
- **Достизање агената који користе различита окружења** – иста способност не би требало да се поново гради за сваки хост. Решено паковањем једног скупа вештина иза дефиниција MCP алата-сервера (у `constitution/mcp/`) и Claude Цоде додатака (у `constitution/plugins/`), тако да се једна имплементација излаже на свим интерфејсима.

Технолошки стек

- **Shell (примарни језик)** — изабран јер алати за инсталацију и регистрацију морају да раде на сваком месту где постоји агент, без потребе за претходним покретањем рунтима-а; покреће `register.sh` и `install_upstreams`, чиме се улазна тачка одржава без зависности и преносивом.
- **Гит подмодули** — изабрани како би се наслеђивало управљање без дуплицирања: Helix Constitution је монтиран на `constitution/` као активна референца, па се ажурирања правила шире преко једног показивача уместо да се копирају и забораве.
- **Model Context Protocol (MCP)** — изабран као стандардни, рунтима-агностички интерфејс алата за агенте; два MCP сервера (медиа-валидатор, `сцхедулед-work`) дефинисана су под `constitution/mcp/` како би вештине биле доступне као позиви алата.
- **Claude Цоде плугинови** — изабрани да би се вештине директно убациле у рунтима агента без додатног кода; два плугина (`helix`, `сцхедулед-work`) испоручују се под `constitution/plugins/`, одражавајући MCP површину за други хост.
- **Поново употребљиви мотори (цонтинуум, токен_оптимизер, сесион_оркестратор, цлицкуп_sync)** — изабрани да би се заједничка механика издвојила из појединачних вештина за поновну употребу у различитим пројектима; токен_оптимизер, на пример, повезан је са пакетима `vasic-digital` (TOOH, Ембеддингс, VectorDB, Нормализе, цонверсатион) и HelixDevelopment-овим LLMProvider преко декларисаних зависности уместо дуплицираног кода.
- **Гит огледало на више хостова (GitHub, GitLab, GitFlic, GitVerse)** — изабрано како би отказ једног хоста или регионална блокада нису могли да прекину приступ; исти

репозиторијум одржава се активним на четири платформе ради отпорности и доступности.

Статус и напомене о искрености

- **Статус: бета.** Седам уставних вештина, два MCP сервера и два плагина испоручени су; нацртане вештине су индексиране и чекају активацију, а три од четири мотора првог нивоа (сесион_орцхестратор, токен_оптимизер, цлицкуп_sync) још увек су у фази дизајна.
- У РЕАДМЕ фајлу пројекат се назива `helix_skills`; канонски GitHub пут је `HelixDevelopment/skills`. Број пронађених налаза у РЕАДМЕ фајлу је самопријављена цифра.

Приоритетни ниво: Helix-primary.